ROTAS API

Esta lista vem do código que você me enviou (ApiHandler e LiveSocket). Se você mudou rotas ou campos ao refatorar, me avise para eu ajustar.

Regras gerais (todas as rotas HTTP)

- Prefixo /api. A sessão usa o cookie qr_session (HttpOnly, 7 dias). O front precisa enviar credentials: "include".
- O corpo é JSON de até 1 MB. Acima disso volta **413**. Se não for um objeto JSON, volta **400**.
- Todo erro volta como { "error": "mensagem" }.
- Uma Origin fora de ALLOWED_ORIGINS recebe **403** “Origem inválida.”.
- Uma rota ou método que não existe recebe **404** “Operação não encontrada.”.
- Uma falha inesperada recebe **500** “Não foi possível concluir. Tente novamente.”.

Autenticação

POST /api/register: cria a conta e já abre a sessão

- **Recebe:** name (3 a 80 caracteres), email, password (8 a 128) e role ("client" ou "provider").
- **Retorna 200:** { "user": { id, name, email, role } } e define o cookie.
- **Erros:** 400 “Informe um e-mail válido e senha de 8 a 128 caracteres.” ou “Confira seu nome e tipo de conta.”; 409 “Este e-mail já está cadastrado.”

POST /api/login: autentica e abre a sessão

- **Recebe:** email e password.
- **Retorna 200:** { "user": { id, name, email, role } } e define o cookie.
- **Erros:** 400 (mesma mensagem de validação do cadastro); 401 “E-mail ou senha incorretos.”

GET /api/me: dados de quem está logado

- Não exige login e não recebe nada além do cookie.
- **Retorna 200:** { "user": {id,name,email,role}, "profile": {...} }.
- profile é o perfil profissional se existir. É null para clientes ou prestadores sem perfil.
- Sem sessão válida, user e profile vêm null.

POST /api/logout: encerra a sessão

- Apaga a sessão no banco e zera o cookie.
- **Retorna 200:** { "ok": true }.

Prestadores

GET /api/providers: lista pública de prestadores

- Não recebe nada.
- **Retorna 200:** { "providers": [{ user_id, name, category, title, bio, city, region, price, unit, phone, lat, lng, radius, updated, rating, review_count }] }.
- A ordem é por nota média (maior primeiro) e depois por updated.
- rating é a média, e vale 0 se não há avaliações.
- O filtro por categoria, região e distância não é feito aqui, então o front precisa filtrar.

POST /api/profile: cria ou atualiza o perfil

- **Exige:** login como provider.
- **Recebe:**
  - category: texto com categorias separadas por vírgula, só entre Eletricista, Encanador, Limpeza, Pintor, Jardinagem, Montador, Ar-condicionado, Reformas.
  - title: até 100 caracteres.
  - bio: até 2000 caracteres.
  - city: até 100 caracteres.
  - region: até 300 caracteres.
  - unit: um de "por serviço", "por hora", "por diária", "a combinar".
  - phone: deve ser 55 + DDD + número (10 ou 11 dígitos depois do 55). Só os dígitos são guardados.
  - price: de 0 a 1.000.000.
  - radius: de 1 a 500, em km.
  - lat e lng: opcionais, mas precisam vir os dois ou nenhum.
- **Retorna 200:** { "ok": true }.
- **Erros:** 401 “Entre na sua conta para continuar.”; 403 “Apenas prestadores podem editar perfis profissionais.”; 400 “Confira os campos. WhatsApp deve incluir 55 + DDD + número. Defina uma região, preço válido e raio de 1 a 500 km.”

Contato e avaliações

POST /api/contact: inicia o contato com um prestador

- **Exige:** login como client.
- **Recebe:** { "id": "<id do prestador>" }.
- Registra que o cliente contatou o prestador, o que libera a avaliação.
- **Retorna 200:** { "url": "https://wa.me/<telefone>?text=<mensagem padrão>" }.
- **Erros:** 401; 403 “Use uma conta de cliente para solicitar atendimento.”; 400 “Identificador do prestador não informado.”; 404 “Prestador não encontrado.”

GET /api/reviews?id=<id do prestador>: avaliações de um prestador

- Não exige login. O cookie é opcional.
- **Retorna 200:** { "reviews": [{ rating, comment, created, name }], "contacted": true|false }.
- contacted é true quando o usuário logado já iniciou contato com esse prestador. Sem login, é false.
- As avaliações vêm da mais recente para a mais antiga.
- **Erro:** 400 “Identificador do prestador não informado.”

POST /api/review: cria ou atualiza a avaliação

- **Exige:** login como client que já usou /api/contact com esse prestador.
- **Recebe:** id (prestador), rating (inteiro de 1 a 5), comment (10 a 1000 caracteres) e confirmed: true.
- Cada cliente tem uma avaliação por prestador, e reenviar atualiza a anterior.
- **Retorna 200:** { "ok": true }.
- **Erros:**
    - 401.
    - 403 “Use uma conta de cliente para avaliar.”
    - 403 “Você precisa iniciar contato antes de avaliar.”
    - 400 “Nota inválida.”
    - 400 “Confirme o atendimento, selecione uma nota e escreva pelo menos 10 caracteres.”
    - 400 “Identificador do prestador não informado.”

WebSocket

ws://host:8080/api/live: atualização da lista de prestadores

- Você envia o texto sync, e o servidor responde { "providers": [...] }, no mesmo formato do GET /api/providers.
- Qualquer outro texto recebe { "error": "Mensagem não reconhecida." }. Se o banco falhar, recebe { "error": "Atualização indisponível" }.
- O servidor só responde quando recebe sync. Não envia nada por conta própria, então o front precisa mandar sync de tempos em tempos ou depois de uma ação.
- Não exige login.

Dois pontos que valem checar

- **Telefone público:** GET /api/providers devolve o phone de todos os prestadores, e o /ws/live também. Isso torna desnecessário o POST /api/contact para obter o número. Só vale ajustar se vocês querem que o contato seja uma ação registrada do cliente.
- **Origem no WebSocket:** a checagem de Origin e o CORS ficam no ApiRoutes. O /api/live é registrado antes, como caminho exato, então não passa por essa checagem.